

(1p)

안녕하세요, 속도는 벡터 팀, 발표 시작하겠습니다. 저희 팀은 이번 학기 동안 vector-augmented Analytical query를 처리할 때, vector index가 부재한 상황에서의 adaptive sampling을 개선하는 연구를 진행했습니다. 먼저 이 문제가 어떤 상황에서 발생하는지 보겠습니다.

(2p)

이 슬라이드는 저희가 다루는 쿼리 형태입니다. vector-augmented analytical query, VAQ. 단어를 들었을 때는 비슷한 벡터 데이터를 찾아라 정도로 이해되죠? 하지만 이 VAQ는 관계형 조건과 벡터 조건이 하나의 SQL 안에 함께 들어갑니다. 예를 들어 고객, 주문 같은 관계형 테이블을 조인 하면서, 동시에 partsupp_deep이라는 벡터 테이블에 대해 “쿼리 부품 벡터와 거리가 0.86보다 작은 부품”을 찾습니다. 이런 쿼리를 VAQ라고 부릅니다. 이때 DB optimizer는 단순히 벡터 검색 결과만 보는 것이 아니라, 관계형 조건과 벡터 조건이 섞인 전체 table join 실행 계획을 결정해야 합니다. 문제는 이때 벡터 조건의 결과 행 수, 즉 카디널리티를 잘못 추정하면 plan 전체가 바뀐다는 점입니다.

(3p)

(영상을 보여주며) 카디널리티는 조건을 만족하는 결과 행 수입니다. 이와 함께 선택도라는 개념도 필요합니다. 선택도는 전체 row 중 조건을 만족하는 row의 비율입니다. 예를 들어 100만 개 row 중 1만 개가 조건을 만족하면, cardinality는 1만이고 선택도는 1%입니다. DB optimizer는 이 값을 보고 index를 사용할지, scan을 할지, join 순서를 어떻게 정할지 결정합니다. 그래서 카디널리티나 선택도 추정이 틀리면 뒤쪽 plan이 완전히 달라질 수 있습니다. 지금 보시는 영상은 같은 쿼리라도 추정값이 달라졌을 때 실행 계획이 어떻게 바뀌는지를 보여주는 예시입니다. 보시는 것처럼 카디널리티 추정이 틀리면 plan과 실행 시간이 모두 달라질 수 있습니다. 그런데 기존 벡터 시스템들은 이 값을 매우 거칠게 추정합니다.

(4p)

기존 시스템의 문제는, 벡터 조건의 선택도를 데이터와 쿼리의 실제 분포와 무관하게 고정 비율로 가정한다는 점입니다. pgvector는 33.3%, VBASE는 50%, DuckDB-vss는 사실상 100%로 선택도를 가정합니다. 최종 cardinality는 이 선택도에 전체 row 수를 곱해서 계산됩니다. 하지만 실제 벡터 조건의 선택도는 query vector의 위치, 거리 임계값, 그리고 데이터 분포에 따라 크게 달라집니다. 어떤 쿼리는 극히 일부 row만 만족하고, 어떤 쿼리는 많은 row가 만족할 수 있습니다. 따라서 데이터와 쿼리 분포를 보지 않는 고정 비율 추정은 잘못된 cardinality 추정으로 이어지고, 결국 optimizer가 비효율적인 plan을 선택하게 만들 수 있습니다. 이 문제를 다룬 관련 연구가 **Exqutor**입니다. Exqutor는 벡터 조건의 cardinality를 더 정확하게 추정해서 optimizer가 더 나은 실행 계획을 고를 수 있도록 하는 방법을 제안했습니다. Exqutor는 index가 있으면 index를 활용하고, index가 없으면 Adaptive Sampling을 사용합니다. 저희 연구는 그중 **index가 없는 경우에** 집중합니다.

(5p)

이 슬라이드는 index가 없을 때 Exqutor가 사용하는 Adaptive Sampling의 흐름입니다. index가 없으면 전체 테이블을 모두 확인하는 비용이 큼니다. 그래서 Exqutor는 전체 데이터 중 일부 sample만 뽑고, 그 sample 안에서 벡터 조건을 만족하는 비율을 계산한 뒤, 전체 cardinality를 근사합니다. 흐름은 네 단계입니다. 첫 번째, 한 쿼리당 표본을 뽑습니다. 여기서는 초기 표본 크기를 385개로 사용합니다. 두 번째, 그 표본 위에서 벡터 조건을 평가해서 cardinality를 추정합니다. 세 번째, 추정 오차를 Q-error로 측정합니다. Q-error는 추정값과 실제값의 비율 오차이고, 1에 가까울수록 좋습니다. 네 번째, 여러 query에서 누적된 Q-error를 보고 다음 구간에서 사용할 표본 크기 N을 조정합니다. 여기서 중요한 점은, 기존 Adaptive Sampling의 핵심이 얼마나 많이 뽑을지, 즉 sample size를 조정하는 데 있다는 것입니다. 저희 연구는 이 no-index Adaptive Sampling 경로에서, sample size가 아니라 표본을 어떻게 선택할지에 집중했습니다. 같은 sample budget 안에서 더 좋은 cardinality 추정을 만들 수 있는 표본 추출 방식이 있는지를 실험했습니다.

(6p)

그래서 저희의 연구 질문은 한 문장입니다. 카디널리티 추정을 더 잘할 수 있는 표본 추출 방식은 무엇인가? 왼쪽 baseline은 기존 방식처럼 균일 무작위 표본을 뽑습니다. 그런데 벡터 데이터는 공간적으로 균일하지 않을 수 있습니다. 어떤 영역은 데이터가 조밀하고, 어떤 영역은 비어 있을 수 있습니다. 그래서 오른쪽처럼 클러스터, 공간 곡선, 해시, 차원 축소 등 다양한 패턴을 이용해 표본을 더 잘 뽑을 수 있는지 탐색했습니다. 다시 말해, 기존 방식이 “무작위로 뽑는다”였다면, 저희는 데이터의 구조를 반영해서 뽑는 방법들이 cardinality 추정에 실제로 도움이 되는지를 비교했습니다.

(7p)

실험에서는 같은 쿼리, 같은 데이터에 대해 세 가지 추정 방식을 비교했습니다. 첫 번째는 baseline 입니다. 기존 Adaptive Sampling 에 가까운 균일 무작위 표본 방식이고, 여기서 cardinality 추정값 하나를 얻습니다. 두 번째는 새 표본 추출 방식을 baseline 대신 단독으로 쓰는 방법입니다. 하지만 method 별로 특정 query 에서 크게 틀리는 경우가 있어, 최종 실험에서는 안정적인 결합 방식을 사용했습니다. 그래서 저희가 사용한 것은 세 번째, 결합 방식입니다. baseline 추정값 하나와 별도 method 의 추정값 하나를 동시에 구하고, 두 값을 산술 평균해서 최종 cardinality 로 사용했습니다. 즉, 저희 실험은 "새 방법이 baseline 을 완전히 대체하는가"가 아니라, baseline 에 구조적인 표본 정보를 결합했을 때 더 좋은 추정이 되는가를 보는 실험입니다.

(8p)

실험은 다섯 개 데이터셋에서 진행했습니다. 이미지 검색 계열의 DEEP 과 SIFT, 이미지 유사도 데이터인 SSN++, 멀티미디어 데이터인 YFCC, 그리고 관계형 TPC-H 의 PartSupp PK 를 포함했습니다. 오른쪽은 조작변인입니다.

첫째, 데이터 크기를 조절하는 scale factor 를 3 가지로 두었습니다.

둘째, 쿼리 조건 폭을 뜻하는 selectivity 도 세 값으로 두었습니다.

셋째, 파라미터 K 를 여러 값으로 테스트했습니다.

넷째, 표본 추출 방식은 baseline 인 Bernoulli 와 분포 인지 방식 13 개, 총 14 개를 비교했습니다.

(9p)

이 슬라이드는 그 13개 표본 추출 방식을 7개 패러다임으로 묶어 보여줍니다. 왼쪽 baseline은 각 row를 같은 확률로 뽑는 Bernoulli 방식입니다. 오른쪽 7개는 이 baseline과 다른 기준들입니다. 공간곡선은 고차원 벡터를 1차원 순서로 펼쳐 뽑고, 차원축소는 PCA나 rSVD처럼 저차원으로 줄인 뒤 뽑습니다. 고전 stratification은 밀도에 따라 계층을 나누고, 양자화는 벡터를 코드나 bin으로 압축한 뒤 뽑습니다. 클러스터링은 비슷한 벡터끼리 묶고, 스트리밍은 데이터를 한 번 훑으며 가중치 기반으로 표본을 유지합니다. 마지막 해시는 hash bucket이나 sketch를 사용합니다. 이제 다음 슬라이드에서, baseline에 이 방법들을 결합했을 때 Q-error가 실제로 개선되는지 보겠습니다.

(10p)

결과를 보겠습니다. 이 슬라이드는 전체 실험 조건에서 baseline 과 결합 방식의 Q-error 를 비교한 결과입니다. 다시 한 번 말씀드리면, Q-error 는 낮을수록 더 정확한 추정입니다. 전체 1,508 개 실험 cell 중에서, 결합 방식이 baseline 보다 더 낮은 Q-error 를 보인 경우가 1,344 개였습니다. 비율로는 89.1%입니다. 여기서 cell 은 데이터 셋, scale factor, selectivity, K, method 같은 조건이 하나로 정해진 비교 단위라고 보시면 됩니다. 즉, 같은 조건에서 baseline 추정과 결합 추정을 비교한 것입니다. 아래 평균 Q-error 를 보면 baseline 은 1.4582, 결합 방식은 1.4019 입니다. 평균적으로 결합 방식이 더 정확한 cardinality 추정을 했다고 볼 수 있습니다. 다음 슬라이드에서는 Q-error 개선이 실제 실행 plan 선택에도 도움이 되는지 확인했습니다.

(11pg)

앞 슬라이드까지는 Q-error, 즉 추정 값 자체의 정확도를 봤습니다. 하지만 optimizer 관점에서 더 중요한 질문은 이것입니다. 정확한 추정이 실제로 더 빠른 plan 선택으로 이어지는가? 그래서 실제 엔진에서 응답 시간을 측정했습니다. 조건은 DEEP, scale factor 10 이고, 총 156 개 plan 의 평균을 비교했습니다. 위쪽 막대그래프를 보시면, pgvector 기본 방식은 평균 5,677ms 가 걸렸습니다. 반면 논문 원본 baseline 은 977.6ms 로, pgvector 기본보다 약 5.81 배 빨랐습니다. 저희 결합 방식은 983.5ms 로 baseline 과 거의 같습니다. 차이는 5.9ms, 비율로는 약 0.6% 정도입니다. 즉, 평균 latency 자체는 baseline 과 사실상 동등하다고 볼 수 있습니다. 그런데 아래쪽의 plan 선택 결과를 보면 차이가 훨씬 큼니다. baseline 은 156 개 중 91 개에서 최적 plan 을 선택했습니다. 비율로는 58.3%입니다. 반면 결합 방식은 156 개 중 148 개에서 최적 plan 을 선택했습니다. 비율로는 94.9%입니다. 여기서 최적 plan 은 실제 실행 시간을 기준으로 가장 좋은 plan 을 의미합니다. 즉, 단순히 추정 값이 좋아졌다는 것뿐만 아니라, 실제 엔진에서 더 자주 좋은 실행 계획을 고르게 되었다는 뜻입니다. 따라서 결합 방식은 평균 latency 를 baseline 보다 나쁘게 만들지 않으면서, 최적 plan 선택률을 58.3%에서 94.9%로 높였습니다. 따라서 저희 방법은 optimizer 의 plan 선택 안정성도 개선했다고 볼 수 있습니다.

(12pg)

마지막으로 향후 연구 방향입니다. 첫 번째는 검증 범위 확장입니다. 현재 본 측정은 특정 scale 과 selectivity 조건을 중심으로 진행되었기 때문에, 앞으로는 더 큰 scale factor, 더 넓은 selectivity, 다중 벡터 조건, 그리고 다른 엔진에서도 결합 효과가 유지되는지 확인할 필요가 있습니다. 두 번째는 History-aware Adaptive Sampling입니다. 실제 서비스에서는 비슷한 query가 반복해서 들어올 수 있으므로, 과거 query의 feedback을 누적하면 현재 query에서 필요한 sampling 비용을 줄일 수 있습니다. 따라서 향후에는 더 넓은 환경에서 결합 효과를 검증하고, 과거 query 정보를 활용해 adaptive sampling 비용을 줄이는 방향으로 확장할 계획입니다.

(13pg)

정리하면, 저희는 vector index가 없는 VAQ 환경에서 Adaptive Sampling의 표본 선택 방식을 개선했습니다. 같은 sample budget 안에서 분포 인지 표본 추출 방식을 baseline과 결합했고, 전체 실험 cell의 89.1%에서 Q-error를 개선했습니다. 또한 실제 엔진 실험에서도 최적 plan 선택률을 58.3%에서 94.9%로 높였습니다.

이상으로 발표를 마치겠습니다. 감사합니다. 질문 받겠습니다.